PACT: Pruned Agent Call Throughput

Smit Vasani
svasani2@illinois.edu

Aditya Prasath Ravilla Shridhar Prasad
apr11@illinois.edu

Chandra Sreeja Chinigepalli
cc215@illinois.edu

Ram Prakash Reddy Motati
rmotati2@illinois.edu

Vanshika Chokhani
vc50@illinois.edu

1 Abstract

2 Introduction

3 Background and Motivation

4 Design

5 Implementation

6 Evaluation

7 Related Work

8 Conclusion

9 Team Member Contributions

10 What is the problem?

Modern autonomous agents in high-stakes domains increasingly rely on iterative planning and self-correction to ensure accuracy [1] [2]. However, this reliability introduces a critical "over-deliberation" bottleneck, where internal reflection latency often dwarfs actual execution time [3]. Current mitigations such as prompt engineering for conciseness or per-token confidence thresholds are insufficient; they are either too brittle to ensure logical consistency or fail to distinguish between linguistic predictability and task-level readiness. Furthermore, rigid constraints like reasoning token budgets, as seen in recent Qwen models, lack the nuance to handle varying task complexities, often resulting in premature truncation on difficult queries while remaining inefficient on simple ones. PACT addresses these failures by decoupling the "exit decision" from the primary model, using a fine-tuned Speculator to dynamically truncate reasoning only when a viable solution has been reached.

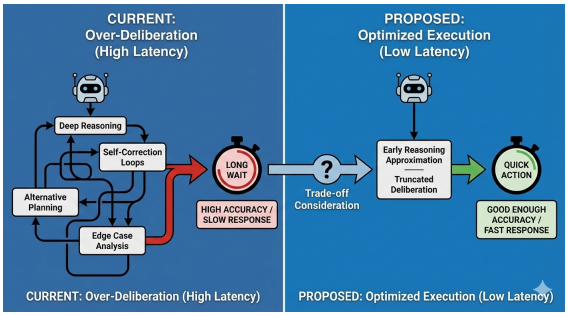


Figure 1: Over Deliberation vs Optimized Execution

11 Why does it matter?

Current LLM driven autonomous agents are fundamentally constrained by a crippling latency bottleneck that threatens to render them impractical for production deployment. Designed to prioritize deep reasoning and task accuracy, these agents frequently fall into a state of "over deliberation," entering prolonged self reflective loops to account for rare or hypothetical edge cases [3]. Because these agents execute sequences of blocking API interactions, their latency accumulates linearly over multi step horizons. Consequently, state of the art systems currently take tens of minutes to hours to complete single runs, such as requiring up to 45 minutes for deep research pipelines [4].

This architecture mistakenly treats perfect correctness as the primary optimization objective, ignoring the strict latency constraints of real world enterprise environments. In speed sensitive applications, particularly in customer support and similar real time interactive environments, responsiveness will frequently be more valuable than perfect correctness. When agents over deliberate, the real world consequences are severe: users will abandon sluggish chat interfaces, and support queues will back up due to multi step tool call delays. Furthermore, as reasoning models continue to scale by incorporating deeper self correction layers, they yield severely diminishing accuracy gains at a rapidly increasing computational cost, making current sequential baselines economically unsustainable

for production scaling.

Ultimately, solving this execution latency will be the critical missing link in shifting agentic AI from a strict focus on research accuracy to actual production responsiveness. To achieve this, our proposed work will establish principled cost latency trade offs by fine tuning dedicated speculator models to independently approximate or truncate unnecessary deliberation. By improving the speculator's accuracy directly, we intend to slash the time to output from minutes down to mere seconds. This optimized execution will make autonomous agents viable for speed critical applications, delivering the essential systems foundation needed to transform computationally expensive academic novelties into highly responsive, enterprise grade applications capable of handling real time customer support workloads at scale.

12 Related Work

12.1 Speculative Decoding and Execution

Speculative decoding [5] accelerates LLM inference by using a smaller draft model to predict tokens verified in parallel by a larger model, achieving throughput gains without quality loss. LayerSkip [6] extends this paradigm by training decoder-only models with a layer-dropout schedule, enabling early exit at intermediate layers and combining it with self-speculative decoding without requiring an auxiliary model. TIDE [7] takes a complementary approach, training lightweight binary MLP classifiers on frozen model hidden states using cosine similarity as a convergence signal, achieving early exit with under 525K additional parameters and no base model retraining. SpecReason [8] extends speculative decoding to reasoning, exploiting the semantic flexibility of thinking tokens: a lightweight model handles simpler intermediate steps while the base model verifies outputs, achieving 1.4–3.0$\times$ speedup with 0.4–9.0% accuracy improvements. Speculative Actions [9] applies the same principle at the agentic action level, parallelizing tool invocations across steps to achieve up to 55% next-action prediction accuracy. While these works accelerate the outer execution loop or target layer-level exit in single-turn generation, PACT targets the inner deliberation loop of multi-step agentic systems, pruning reasoning chains before any tool call is issued, and delegates the exit decision to a dedicated fine-tuned Speculator rather than relying on the primary model's own hidden states.

12.2 Agentic Systems and Over-Deliberation

Autonomous agents iteratively plan, reflect, and self-correct [1, 2], but this reliability introduces severe over-deliberation [3]. Because agents execute blocking API calls sequentially, latency accumulates linearly, with state-of-the-art pipelines requiring up to 45 minutes per run [4]. Chain-of-Thought prompting [10], while foundational to enabling such reasoning, is itself a root cause of this bottleneck: decomposing problems into intermediate steps incurs significant token overhead that compounds across multi-step agentic horizons. Recent empirical work [11] quantifies this precisely, demonstrating that task accuracy follows an inverted U-shaped curve with reasoning chain length, and that easy problems begin to over-deliberate as early as 2K tokens while hard problems require up to 8K, confirming that uniform compute allocation is fundamentally wasteful. Flow-of-Options [1] and EXACT [2] enrich reasoning for correctness by diversifying trajectories and enabling reflective self-correction; PACT takes the complementary stance, learning to recognize when sufficient reasoning has already occurred and exiting early. ToolBench [12] informs our data collection by providing diverse real-world tool-use trajectories from which we label viable early exit points. The ReAct framework [13], which interleaves reasoning traces with tool actions in an iterative Thought $\rightarrow$ Action $\rightarrow$ Observation loop, defines the agent execution paradigm that PACT directly targets for latency optimization.

12.3 Reasoning Budgets, Adaptive Exit Policies, and Hybrid Architectures

The hybrid SAS-MAS paradigm of [14] addresses reasoning cost at the *request level*: an LLM-based rater routes each query to either a fast single-agent system or a robust multi-agent system, achieving 1.1–12% accuracy improvements with up to 88.1% cost reduction. However, this constitutes *macro-efficiency*: once a request is routed to a deep-reasoning agent, that agent remains bound by sequential, blocking deliberation. PACT instead performs *intra-execution optimization*, with the Speculator monitoring the live reasoning trace and triggering early exit at confidence threshold τ.

Recent frontier models such as Qwen3 [15] introduce hard reasoning token budgets to curb over-deliberation, while Token-Budget-Aware LLM Reasoning [16] proposes dynamically estimating a per-instance token budget before generation begins. Both approaches are fundamentally *input-bound*: they set a ceiling on deliberation prior to execution, without monitoring whether the reasoning trace has actually converged. This produces two symmetric failure modes: a simple query that converges early but

holds a large budget wastes the intervening computation, while a complex query still mid-deliberation at budget exhaustion is forced to emit a tool call from an incomplete reasoning state. Dynamic Early Exit [17] partially addresses this by monitoring model behavior at reasoning transition points such as "Wait" tokens and terminating generation when confidence is high, achieving 31–43% CoT length reduction without any fine-tuning. Ada-RS [18] improves on static budgets by framing the stopping decision as a learned adaptive policy. PACT goes further than all of these, removing the budget abstraction entirely and replacing it with a *convergence-bound* exit policy: the Speculator stops because it *knows enough*, not because it *ran out of space* or hit a pre-estimated ceiling, with a single threshold τ governing behavior across heterogeneous tasks without per-task ceiling calibration.

12.4 Reasoning Chain Pruning and Overthinking Mitigation

A growing body of work targets redundant deliberation in reasoning chains. TERMINATOR [19] learns optimal exit points by supervising on the first arrival of a model's correct final answer, achieving 14–55% CoT length reduction across MATH-500, AIME 2025, HumanEval, and GPQA. "Don't Overthink It" [20] demonstrates that shorter reasoning paths are more effective because they involve fewer backtracks, and that a short-$m@k$ selection strategy can reduce compute by up to 40% while matching majority-voting performance. Sui et al. [21] provide a taxonomy of overthinking in large reasoning models, categorizing mitigation strategies into post-training, prompt-based, and output-based approaches, establishing the conceptual vocabulary that motivates PACT's design. Confident Adaptive Language Modeling (CALM) [22] introduced the foundational idea of confidence-threshold early exit, achieving 2–3$\times$ speedup on translation tasks, which PACT extends to the agentic tool-call setting. PACT differentiates from all of the above by decoupling the pruning decision entirely from the primary model, delegating it to a fine-tuned Speculator that monitors the primary model's reasoning stream independently. This architectural separation allows PACT to apply pruning without modifying the primary model and to optimize the exit policy via DPO preference alignment [23], constructing preference pairs of pruned trajectories (chosen: earliest correct exit) against over-deliberated ones (rejected: full reasoning chain). This transforms the Speculator from a simple classifier into a latency-optimized policy, enabling principled accuracy-latency tradeoffs that go beyond heuristic truncation or input-side budget estimation.

13 Proposed Solution

13.1 Data Collection and Labeling

13.1.1 Dataset Source and Role

We ground our work in ToolBench [12], a large-scale corpus for tool use and function calling hosted as `tuandunghcmut/toolbench v1`. The corpus is structured as instruction-style supervision through multi-turn dialogs where a model must compose `Thought` $\rightarrow$ `Action` $\rightarrow$ `Action Input` sequences and consume function observations. This design targets generalization across diverse real world APIs rather than simple "gadget" demonstrations, matching our need for realistic task text, structured calls, and noisy success/failure modes encountered in production.

We utilize ToolBench in two complementary ways:

- **Benchmark Splits:** Instruction configurations (G1/G2/G3) provide tasks alongside comprehensive API specifications, including names, descriptions, and parameter schemas, allowing the agent to execute against a task-specific tool surface.
- **Training Conversations:** The default configuration provides multi turn dialogs that serve as a source for ground truth solution paths. From these, we extract ordered steps with natural language reasoning paired with faithful tool-argument pairs, capturing "what was said" versus "what was called" without manual scenario drafting.

13.1.2 Data Processing Pipeline

Our parser maps raw ToolBench rows into an internal task object containing the prompt, available tool specs, and a `ground_truth_path`. Because raw dialogs are not direct classification labels, we derive a supervised set for the Speculator a model designed to estimate from partial reasoning whether it is appropriate to commit to a specific call via the following pipeline:

Trajectory Sources Trajectories are sourced through a three-pronged approach. First, tasks with existing ground-truth paths in ToolBench are labeled directly using published thoughts and actions. Second, for benchmark tasks lacking paths, we deploy a ReAct style agent [13] to explore the task. Third, we grade these exploratory runs using a hybrid of reference path comparison and LLM-based judgment, retaining only successful trajectories to ensure the Speculator learns from optimal behavior.

Cumulative Reasoning Prefixes To simulate the real time stream of an LLM's thought process, we take the thought text for each step, perform sentence splitting,

and construct monotone prefixes. For instance, a thought containing three sentences is decomposed into three separate training instances: the first sentence, the first two combined, and the full thought. Each prefix represents a distinct "reasoning-so-far" state along a trajectory toward the recorded action.

Oracle-Based Labels For every cumulative prefix, we must determine if the reasoning contains sufficient grounding to justify the target tool name and specific argument values. We utilize a frontier LLM (GPT-5) as a labeling oracle to avoid the brittleness of deterministic string-matching, which often fails when tool names are non-lexical or arguments are paraphrased. The oracle receives a fixed prompt defining its role and must return a binary (`YES` or `NO`) verdict. A `YES` indicates the prefix is "commit-ready," while a `NO` signals that the model must continue accumulating reasoning.

Convergence-Point Identification We scan the labeled prefixes chronologically and apply a stopping rule, where we only assign a positive label to the *earliest* accepted prefix in a sequence. This ensures the Speculator is trained to recognize the first defensible point of convergence, specifically targeting the reduction of over-deliberation latency. The final dataset is exported in JSONL format, mapping each task prompt and thought prefix to its binary convergence label.

13.2 Fine-Tuning Strategy

The PACT framework employs a staged fine-tuning pipeline to transform a compact language model into an efficient Speculator. We initialized our baseline Speculator using the frozen weights of the Phi-3-mini architecture, augmenting it with a task-specific linear classification head. This "head-only" tuning approach ensures the model retains its pre-trained linguistic reasoning while learning to map internal states to a binary "readiness" signal. The model was trained via Supervised Fine-Tuning (SFT) on a curated dataset of 3,000 agent trajectories derived from ToolBench [12], with 2,850 examples allocated for training over two epochs.

Our initial evaluation on a validation set of 150 samples reveals a significant calibration gap. As shown in Figure 2, while the model achieves a peak accuracy of 81.3%, this performance is only reachable at a low confidence threshold ($\tau \approx 0.26$). At the standard classification threshold of $\tau = 0.5$, the Speculator behaves cautiously, achieving high precision (0.887) but poor recall (0.724). Conversely, optimizing for recall (0.897) at the lower threshold introduces 19 false-positive "READY" signals, which in a production environment would lead to premature and likely malformed tool invocations. These results, summarized in Table 1, indicate that the SFT baseline internalizes the "Aha!" moment but lacks the probabilistic confidence to trigger it efficiently.

To move beyond this cautious baseline, our next phase involves doing full-scale SFT and scaling it to a larger trajectory corpus to improve the model's foundational grasp of diverse tool schemas. Following this, we will implement Direct Preference Optimization (DPO) to explicitly refine the model's confidence distribution [23]. By providing the Speculator with preference pairs that favor pruned, successful trajectories over redundant, over-deliberated ones, we intend to "stretch" the confidence curve. This alignment will penalize the observed under-confidence and push the model toward a policy that is both "confidently fast" and precise, effectively resolving the current accuracy-latency trade-off [18].

Table 1: SFT Baseline Performance: Cautious vs. Aggressive

Threshold (τ)	Acc.	Prec.	Rec.	F1
0.50 (Default)	0.787	0.887	0.724	0.798
0.26 (Best Acc)	0.813	0.804	0.897	0.848

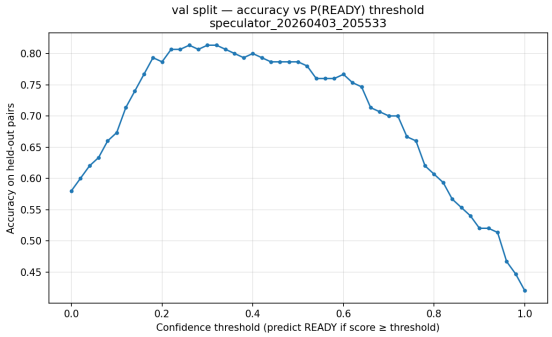


Figure 2: Accuracy vs. Confidence Threshold for the Phi-3-mini Speculator baseline.

14 Evaluation Plan

We evaluate PACT along three dimensions: latency reduction, accuracy under early exit decisions, and pruning effectiveness. All experiments are conducted on ToolBench [12] trajectories spanning tool use tasks of varying complexity. Our goal is to quantify how effectively the fine tuned Speculator learns to predict valid early exit points, and to measure the latency gains achieved through pruning in the subsequent DPO phase.

1. Latency Reduction. Let T_{seq} denote execution time of the baseline agent and $T_{\text{PACT}}(\tau)$ denote execution time when early exit is triggered at confidence threshold τ. We

define relative latency improvement as:

$$\Delta_T(\tau) = \frac{T_{\text{seq}} - T_{\text{PACT}}(\tau)}{T_{\text{seq}}}.$$

We plan to measure end-to-end completion time, time to first tool invocation, and time to first user visible output once the DPO and pruning phase is complete. These metrics would capture how early exit decisions reduce cumulative reasoning latency across multi step workflows.

2. Accuracy, Precision, and Recall under Early Exit. Let A_{seq} denote baseline task accuracy and $A_{\text{PACT}}(\tau)$ denote accuracy under early exit. We define accuracy degradation as:

$$\Delta_A(\tau) = A_{\text{seq}} - A_{\text{PACT}}(\tau).$$

Beyond aggregate accuracy, we track precision and recall of the Speculator's exit predictions separately, as they capture two distinct failure modes. High precision but low recall (as observed in our baseline at $\tau = 0.5$, where precision $= 0.887$ and recall $= 0.724$) reflects an overly cautious Speculator that under predicts readiness, missing valid exit points. Conversely, high recall but low precision (as seen at $\tau = 0.26$, where recall $= 0.897$ but 19 false positive `READY` signals were observed) reflects an over eager Speculator that incorrectly predicts readiness on incomplete reasoning states, risking malformed tool invocations. We sweep τ across both SFT and SFT+DPO models to characterize how DPO shifts this precision-recall tradeoff.

3. Pruning Effectiveness. To directly measure how aggressively the Speculator prunes unnecessary deliberation, we track the exit position within the reasoning trajectory. Let L_{full} denote full reasoning chain length and $L_{\text{exit}}(\tau)$ denote the token position at which exit is triggered. We define pruning ratio as:

$$P(\tau) = 1 - \frac{L_{\text{exit}}(\tau)}{L_{\text{full}}}.$$

Higher $P(\tau)$ indicates more aggressive pruning. We report this metric for the SFT+DPO model to validate that training successfully shifts exit decisions toward earlier valid convergence points in the reasoning trajectory.

15 Team Member Contributions

- *svasani2*: **Current:** Ideation and SFT on linear layer + preliminary evaluation; **Future:** SFT on full model and employ DPO for pruned reasoning.
- *apr11*: **Current:** Data section architecture (ToolBench, and Oracle framing); **Future:** Repository artifact alignment and cross-section consistency.
- *cc215*: **Current:** Surveyed Prior Work and researched Speculator training data; **Future:** Contribute to speculator pruning effectiveness analysis.
- *rmotati2*: **Current:** Data collection, cleaning, and relabeling pipelines for Speculator fine tuning; **Future:** Evaluation framework design
- *vc50*: **Current:**Literature review for related work, including comparative architecture analysis; **Future:** Collaborate evaluation framework design

References

[1] Lakshmi Nair, Ian Trase, and Mark Kim. Flow-of-options: Diversified and improved llm reasoning by thinking through options. In *International Conference on Machine Learning (ICML)*, 2025.

[2] Xiao Yu, Baolin Peng, Vineeth Vajipey, Hao Cheng, Michel Galley, Jianfeng Gao, and Zhou Yu. Exact: Teaching ai agents to explore with reflective-mcts and exploratory learning. In *International Conference on Learning Representations (ICLR)*, 2025.

[3] T. Masterman et al. Ai agent systems: Architectures, applications, and evaluation. *arXiv preprint arXiv:2601.01743*, 2025.

[4] Gregoire Mialon, Roberto Dessì, Maria Lomeli, Christos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, et al. Gaia: a benchmark for general ai assistants. *arXiv preprint arXiv:2311.12983*, 2023.

[5] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.

[6] Mostafa Elhoushi, Akshat Shrivastava, et al. Layerskip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*, 2024.

[7] Tide: Token-informed depth execution for per-token early exit in llm inference. *arXiv preprint arXiv:2603.21365*, 2025.

[8] Rui Pan, Yinwei Dai, Zhihao Zhang, Gabriele Oliaro, Zhihao Jia, and Ravi Netravali. Specreason: Fast and accurate inference-time compute via speculative reasoning. *arXiv preprint arXiv:2504.07891*, 2025.

[9] Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, and Tianyi Peng. Speculative actions: A lossless framework for faster ai agents. In *International Conference on Learning Representations (ICLR)*, 2026.

[10] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022.

[11] When more thinking hurts: Overthinking in llm test-time compute scaling. *arXiv preprint arXiv:2604.10739*, 2025.

[12] Yujia Qin, Shihao Liang, Yining Ye, et al. Toolbench: Facilitating tool use for large language models. *arXiv preprint arXiv:2307.16789*, 2023.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.

[14] Mingyan Gao, Yanzi Li, Banruo Liu, Yifan Yu, Phillip Wang, Ching-Yu Lin, and Fan Lai. Single-agent or multi-agent systems? why not both? *arXiv preprint arXiv:2505.18286*, 2025.

[15] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[16] Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. Token-budget-aware LLM reasoning. *arXiv preprint arXiv:2412.18547*, 2024.

[17] Dynamic early exit in reasoning models. *arXiv preprint arXiv:2504.15895*, 2025.

[18] Suraj Nair et al. Ada-rs: Adaptive rejection sampling for selective thinking in agentic workflows. *arXiv preprint arXiv:2602.04152*, 2026.

[19] TERMINATOR: Learning optimal exit points for early stopping in chain-of-thought reasoning. *arXiv preprint arXiv:2603.12529*, 2026.

[20] Don't overthink it. preferring shorter thinking chains for improved LLM reasoning. *arXiv preprint arXiv:2505.17813*, 2025.

[21] Yang Sui et al. Overview of overthinking in large reasoning models. *arXiv preprint*, 2025.

[22] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. In *Advances in Neural Information Processing Systems*, 2022.

[23] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, 2024.